

Appendix

1.1 Problem Understanding & Use-Case Validation:-

We aim to build a Real Time AI Investment Decision system. This system would predict future price movements, suggest buying and selling, based on live market data, technical analysis, incremental ML models and sentiment analysis. The problem we are solving is relevant because:

- Lack of personalized intelligence: Most trading apps just provide raw charts/metrics and not analyze them in real time, and predict the future movements
- There is so much information related to stocks, like current pricing, indicators and news events that it takes a lot of time for retail investors to analyse all this information, leading to delayed decision making.

Few evidences for the same are:

- Studies from JP Morgan, CFA have shown that using algorithms and rule based signals for trading is much more beneficial than manual trading.
- SEBI reported that over 90% of retail traders lose money, due to bad timing of entry/exit, emotional trading etc.

In recent times companies have started using machine learning and natural language processing (NLP) to understand market behavior more deeply. Platforms like QuantConnect, Alpaca, and Bloomberg Terminal provide data-driven trading insights, while frameworks such as “FinRL: Deep Reinforcement Learning Framework for Automated Trading” have demonstrated how machine learning and reinforcement learning can enhance market predictions. Similarly, models like ARMA, GARCH, and XGBoost have become industry benchmarks for predicting stock trends and volatility. However, these systems still struggle with a few big challenges, mainly real-time processing, scalability, and explainability. This is why we chose to use Pathway for our project as it allows us to build a real-time, efficient, and intelligent investment decision system. Most of these traditional systems work in a batch mode. While this approach works for long-term predictions, it fails to react quickly to fast market movements, which can make all the difference in real-world trading.

1.2 Solution Overview

Streaming is essential for our problem because we need real time stock price and news data, so that our user can act upon our tip as soon as possible, thereby maximising gains and minimising losses. Pathway plays a crucial role in pre-processing as well as stateful computations. Pathway allows incremental updates based on newly ingested ticks and news, avoiding expensive recomputation. Also Pathway can pre-compute all the deterministic computations, which reduces the load on LLM. We will also be using Pathway’s persistence to again restart and load back data, if the power goes off.

Deterministic components of our system include technical analysis (computing metrics like RSI, MACD etc to understand the stock's history). Time series forecasters like ARMA, GARCH and ML models like XGBoost are used to understand the trends in a better way. LLM agent is used to understand the results of all the computations, to handle conflicting signals, and generate natural language tips and reasoning for the users.

The first step of our system is streaming real time stock prices and news data from various APIs, based on general market trends. The second step is calculating various metrics related to the stock, and also applying some time series models. The third step includes inserting all the metrics into the XGBoost model to get the final predicted change in price. Then using this predicted change of price and initial calculated metrics to suggest buying/selling price for that stock, whenever a big change in stock price is expected.

1.3 User Interface: The Mobile App + The on-device AI Agent

The mobile app (still under development) provides for the user to access the alerts in a personalized manner. When the app first loads, the user is asked to submit his portfolio details and his preferences. This is for the onboarding part of the application.

Now whenever the app receives 'alert' data via push notifications from the backend service (through the Firebase Cloud Messaging), the OS will wake up the app in the background. At this time, an on-device AI agent will be fed the user's preferences, portfolio and the received data. The notification is only sent to the mobile device till now. The notification banner is not yet displayed to the user.

If the ticker/symbol of the data satisfies at least one of the following conditions: (i) It comes under user's preferences (ii) It can impact (positively or negatively) the user with huge amount of money

Then, the AI Agent would try to create a suggestion for the user (like SELL Your AAPL shares for \$x or BUY NVDA for \$y) with good reasoning (as the alert also includes news headlines) and then the agent will now actually notify the user about this alert.

On the home screen of the application, the user will be shown the last few suggestions generated for him/her (the ones received in the last 12 hours), so that in case the user missed any of them he/she can act upon it.

Users will also be facilitated to update their portfolio and preferences.

All the data- the portfolio, the preferences, and the tips/suggestions of the last 12 hours will be stored in Async Storage (internal storage- SQLite database for Android) on the user's mobile device itself.

1.4 Implementation Details and Production Bar

Streaming Ingest & Live Indexing

We built our system to handle real-time financial data as it arrives, keeping latency low while making sure everything stays reliable and consistent.

Kafka-Based Streaming Pipeline

Here's how the data flows through our system:

- Stock ticks and news come in continuously through lightweight producer services that feed into Kafka topics.
- Pathway's Kafka connectors pick up these streams and turn them into dynamic tables that update in real-time.
- Every time a new tick arrives, we incrementally update all the dependent calculations—technical indicators, sentiment scores, ARMA/GARCH forecasts, and feature vectors for our XGBoost model.
- Thanks to Pathway's incremental computation engine, we can update metrics in under a second without having to recalculate entire time windows from scratch.

Live Indexing & Storage

Once we've computed the features, here's what happens next:

- We batch the computed features periodically and write them to ClickHouse, which serves as both our historical data store and the source for training ML models.
- ClickHouse's columnar design with time-series partitioning lets us read and write data quickly, making backtesting much more efficient.

Resilience Measures

We make sure that expensive operations like model inference or LLM reasoning only run after the basic computations are done and validated, which reduces the chance of cascading failures.

Observability

We set up monitoring to keep tabs on system health and catch issues early. We have implemented structured logging through Python and Pathway to track ingestion status, latency, and failures

Testing & Guardrails

Before anything goes live, we validate it thoroughly:

- Simulation runs on historical data to validate signals before they hit production
- Schema validation on Kafka payloads to catch malformed data early
- Safety checks for user-facing signals: anything below our confidence threshold automatically gets converted to a "Hold" recommendation

1.5 List of Technical Metrics Calculated

- **Volatility & Risk Metrics (ARMA + GARCH):** prev_close (Previous closing price), ret (Log return), sigma_forecast (Forecasted volatility), sigma_t (Current volatility), resid (ARMA residual), arma_forecast (ARMA(1,1) forecast), risk_adj_ret (Risk-adjusted return)
- **Momentum Indicators:** rsi_timing (RSI-based timing signal indicating oversold/overbought zones)
- **Moving Averages (EMA):** ema_12, ema_26, ema_20, ema_50, ema_200
- **MACD Indicators:** macd (MACD line), signal (Signal line), histogram (MACD histogram), histogram_prev (Previous histogram value), histogram_growing (Histogram increasing), histogram_shrinking (Histogram decreasing), macd_signal (Classified MACD signal -2 to +2)
- **Trend Filters:** ema_trend_filter_trend_up (Medium-term uptrend), ema_trend_filter_trend_down (Medium-term downtrend), long_term_bias_trend_up (Long-term uptrend), long_term_bias_trend_down (Long-term downtrend)
- **Trading Signals:** long_signal (Buy signal), short_signal (Sell signal)
- **Price Change:** pct_change (Percentage change from previous close)

1.6 Introduction

This project focuses on building a real-time AI Investment Decision system using the pathway framework to address the limitations of traditional, batch-oriented financial systems. The core problem is the high latency and cost associated with processing continuous, real-time stock market and news data, which leads to delayed insights, missed trading opportunities, and potential for retail investors.

Our solution is a production-grade system designed to ingest live data streams, analyze them continuously, and generate immediate, personalized investment tips. The system inputs the user's current portfolio, predicts future price movements, and suggests buying-selling tips by using :

- Real-time stock price and news data streaming.

- The pathway framework for incremental updates and stateful computations, ensuring the system is always up-to-date and minimizing expensive recomputations
- Pre-computation of deterministic components, like technical analysis, to reduce the load on the LLM agent